

Model-Based vs Reinforcement Learning Control for Spacecraft Attitude Stabilization

Kirin Kawamoto
Aerospace Engineering Science
University of Colorado Boulder
Boulder, USA
Kirin.Kawamoto@colorado.edu

Abstract—Spacecraft attitude stabilization is a nonlinear control problem with strict performance requirements. Classical methods such as the Linear Quadratic Regulator (LQR) provide efficient and analytically tractable solutions, while reinforcement learning methods such as Soft Actor-Critic (SAC) offer a data-driven alternative for handling nonlinear dynamics.

This work compares LQR and SAC for rigid-body spacecraft attitude stabilization using Modified Rodrigues Parameters (MRPs) and angular velocity states. The LQR controller is designed from a linearized model using Bryson’s rule, while the SAC agent is trained in a Gymnasium-based simulation environment. Performance is evaluated using return, final attitude error, angular velocity convergence, and success rate.

Results show that LQR provides reliable and efficient stabilization with high precision, while SAC demonstrates the ability to learn a stabilizing policy directly from nonlinear simulation but requires greater training complexity. These results highlight key trade-offs between model-based and learning-based approaches for spacecraft attitude control.

I. INTRODUCTION AND BACKGROUND

Spacecraft attitude stabilization is a foundational problem in aerospace engineering, enabling mission-critical functions such as Earth observation, inter-satellite communication, and deep-space navigation. These applications demand precise orientation control under constraints including actuator limits, external disturbances, and highly nonlinear rotational dynamics. As spacecraft missions become increasingly autonomous and operate in uncertain environments, there is growing interest in control strategies that can maintain performance beyond nominal operating conditions.

This is particularly relevant for spacecraft systems, where high-fidelity models exist but operating conditions may deviate significantly from linearized regimes, raising the question of whether learning-based controllers offer meaningful advantages.

Classical control methods, particularly the Linear Quadratic Regulator (LQR) introduced in [1], provide optimal state-feedback laws for linear systems with quadratic cost functions. When applied to spacecraft attitude dynamics, LQR is typically designed using a linearization about a desired equilibrium and tuned via heuristics such as Bryson’s rule [2]. These approaches are computationally efficient and offer strong theoretical guarantees, but their performance can degrade under large-angle maneuvers or when nonlinear coupling effects dominate the dynamics. As a result, LQR-based controllers

often require gain scheduling or multiple linearizations to maintain robustness across the full state space.

Recent advances in reinforcement learning (RL) offer an alternative paradigm in which control policies are learned directly through interaction with the system. In particular, the Soft Actor-Critic (SAC) algorithm proposed by [3] has demonstrated strong performance in continuous control tasks due to its off-policy formulation and entropy-regularized objective, which improves exploration and training stability. Unlike classical controllers, RL methods do not require explicit linearization and can, in principle, capture nonlinear dynamics and complex control trade-offs through data-driven learning.

The application of RL to spacecraft control has gained increasing attention in recent literature. For example, [4] demonstrate the use of deep RL for autonomous spacecraft attitude control under uncertainty, showing improved robustness compared to traditional controllers in simulation. Similarly, [5] explore policy search methods for nonlinear control problems, highlighting the potential of learning-based approaches in dynamic systems. More recently, [6] introduce the BSK-RL framework, which provides high-fidelity simulation environments for training RL-based spacecraft controllers, emphasizing modularity and scalability for complex mission scenarios. These works suggest that RL can complement or potentially outperform classical methods in regimes where modeling assumptions break down.

While reinforcement learning has demonstrated promise for nonlinear control, its performance remains sensitive to reward design, training dynamics, and sample efficiency. These considerations motivate a direct comparison with classical control methods under consistent conditions.

Accordingly, this work performs a systematic comparison between a classical optimal control method (LQR) and a modern RL algorithm (SAC) for spacecraft attitude stabilization. The spacecraft is modeled as a rigid body using Modified Rodrigues Parameters (MRPs) and angular velocity states, providing a compact attitude representation well suited for large-angle maneuvers when used with shadow-set switching. An LQR controller is implemented as a baseline using analytically derived gains, while a SAC-based controller is trained using curriculum learning within a Gymnasium-compatible environment.

The primary contributions of this work are:

- Implementation of an LQR controller using first-principles linearization and Bryson's rule for systematic tuning
- Development of a SAC-based controller trained directly on nonlinear spacecraft dynamics
- Quantitative comparison of performance, stability, and robustness across multiple evaluation metrics

By evaluating these fundamentally different control paradigms under a unified framework, this study provides insight into the trade-offs between model-based optimal control and data-driven learning approaches for spacecraft attitude stabilization.

A. Change from Proposal

The original proposal considered spacecraft station-keeping as a continuous-action MDP using BSK-RL/Basilisk, with orbital position and velocity errors as the state and bounded thrust as the action. During development, the project was revised to focus on spacecraft attitude stabilization instead. This change was made because the original station-keeping problem introduced substantial simulation complexity and long SAC training times, making it difficult to complete a controlled comparison within the project timeline.

The revised attitude-control problem preserves the main objective of the proposal: comparing a classical model-based controller with a reinforcement learning controller for spacecraft control. Using rigid-body attitude dynamics with Modified Rodrigues Parameters and angular velocity states allowed for a simpler and more transparent LQR design, faster simulation, more manageable SAC training, and a clearer comparison between LQR and SAC performance.

II. PROBLEM FORMULATION

The objective of this work is to stabilize the attitude of a rigid spacecraft to a desired reference orientation. Attitude refers to the orientation of the spacecraft body frame relative to a reference frame. In this work, attitude is represented using Modified Rodrigues Parameters (MRPs) [7], a three-dimensional vector representation of rotation that provides a minimal and nonsingular parameterization for moderate rotations.

The system state is defined as:

$$\mathbf{x} = [\boldsymbol{\sigma}, \boldsymbol{\omega}] \in \mathbb{R}^6 \quad (1)$$

where $\boldsymbol{\sigma} \in \mathbb{R}^3$ represents the MRPs describing the attitude error, and $\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity of the spacecraft expressed in the spacecraft's body frame.

A. Dynamics

The spacecraft dynamics consist of rotational kinematics and rigid body dynamics. The MRP kinematics are given by:

$$\dot{\boldsymbol{\sigma}} = \frac{1}{4} B(\boldsymbol{\sigma}) \boldsymbol{\omega} \quad (2)$$

where $B(\boldsymbol{\sigma})$ is a nonlinear matrix function of the MRPs.

The rotational dynamics follow Euler's rigid body equation:

$$I\dot{\boldsymbol{\omega}} = \mathbf{u} - \boldsymbol{\omega} \times (I\boldsymbol{\omega}) + \mathbf{d} \quad (3)$$

where I is the spacecraft inertia matrix, $\mathbf{u}$ is the control torque, and $\mathbf{d}$ represents external disturbances.

B. Reinforcement Learning Formulation

The control problem is formulated as a Markov Decision Process (MDP). The state consists of the attitude error and angular velocity:

$$s = (\boldsymbol{\sigma}, \boldsymbol{\omega}) \quad (4)$$

The action is a bounded control input:

$$a \in [-1, 1]^3 \quad (5)$$

which is scaled to a physical control torque:

$$\mathbf{u} = u_{\max} \cdot a \quad (6)$$

The reward function penalizes attitude error, angular velocity, and control effort:

$$r = - (q_{\sigma} \|\boldsymbol{\sigma}\|^2 + q_{\omega} \|\boldsymbol{\omega}\|^2 + r_u \|\mathbf{u}\|^2) \quad (7)$$

with additional shaping terms to encourage rapid convergence and provide terminal bonuses upon success.

C. Termination Criteria

An episode is considered successful if both the attitude error and angular velocity fall below specified tolerances:

$$\|\boldsymbol{\sigma}\| < \sigma_{\text{tol}}, \quad \|\boldsymbol{\omega}\| < \omega_{\text{tol}} \quad (8)$$

Episodes terminate either upon reaching the maximum simulation time or, optionally, upon satisfying the success condition.

III. METHODS

Two control approaches are considered: a model-based Linear Quadratic Regulator (LQR) and a model-free reinforcement learning controller based on Soft Actor-Critic (SAC). The LQR controller serves as a baseline derived from a linearized model, while SAC operates directly on the nonlinear system dynamics.

A. LQR Baseline

As a baseline controller, a Linear Quadratic Regulator (LQR) is constructed using a linearized approximation of the spacecraft dynamics. The nonlinear system described in Section II is linearized about the equilibrium point corresponding to zero attitude error and zero angular velocity.

The resulting linear time-invariant system is expressed as:

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u} \quad (9)$$

where $\mathbf{x} = [\boldsymbol{\sigma}, \boldsymbol{\omega}]$ and $\mathbf{u}$ is the control torque.

The LQR control law is obtained by minimizing the infinite-horizon quadratic cost function:

$$J = \int_0^{\infty} (\mathbf{x}^T Q \mathbf{x} + \mathbf{u}^T R \mathbf{u}) dt \quad (10)$$

where $Q \succeq 0$ and $R \succ 0$ are state and control weighting matrices.

The optimal control input is given by:

$$\mathbf{u} = -K\mathbf{x} \quad (11)$$

where the feedback gain matrix K is computed as:

$$K = R^{-1}B^TP \quad (12)$$

and P is the solution to the continuous-time algebraic Riccati equation (ARE):

$$A^TP + PA - PBR^{-1}B^TP + Q = 0 \quad (13)$$

The weighting matrices Q and R are selected using Bryson's rule [2], which normalizes each state and control input by its maximum expected value. This results in diagonal weighting matrices where each entry reflects the relative importance of the corresponding state or control component.

Since the LQR controller is derived from a linearized model, its performance is expected to be optimal only near the equilibrium point. For larger attitude deviations, the nonlinear dynamics may degrade controller performance.

In practice, the resulting control input is applied to the nonlinear system dynamics for evaluation.

B. Soft Actor-Critic (SAC)

To address the limitations of model-based control, a reinforcement learning approach based on Soft Actor-Critic (SAC) [3] is employed. SAC is an off-policy actor-critic algorithm designed for continuous control problems, which extends standard formulations by incorporating entropy maximization into the objective.

Unlike traditional actor-critic methods that maximize expected return, SAC optimizes a stochastic policy by maximizing both expected reward and policy entropy:

$$J(\pi) = \mathbb{E} \left[\sum_t r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right] \quad (14)$$

where α is a temperature parameter that determines the tradeoff between reward maximization and exploration.

The algorithm maintains:

- A stochastic policy $\pi_\phi(a|s)$ (actor)
- Two Q-functions $Q_{\theta_1}(s, a), Q_{\theta_2}(s, a)$ (critics)
- A replay buffer for off-policy learning

At each update step, transitions are sampled from the replay buffer to update the Q-functions using a soft Bellman backup, while the policy is updated to maximize the expected Q-value and entropy. The use of two Q-functions mitigates overestimation bias, and target networks are updated using a soft update rule:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta' \quad (15)$$

1) *Entropy Temperature*: The entropy temperature α plays a critical role in balancing exploration and exploitation. In this implementation, α is adjusted automatically by optimizing:

$$J(\alpha) = \mathbb{E} [-\alpha \log \pi(a|s) - \alpha \mathcal{H}_{\text{target}}] \quad (16)$$

Following common practice, the logarithm of the temperature parameter is optimized instead of α directly to improve numerical stability.

2) *Implementation Details*: The implementation follows the Stable-Baselines3 framework [8]. In contrast to the original SAC formulation, an entropy coefficient is used, which is equivalent to the inverse of the reward scaling factor. This formulation improves numerical stability by preventing excessively large gradients during Q-function updates.

Algorithm 1 Soft Actor-Critic

- 1: Initialize $\theta_1, \theta_2, \phi, \alpha$
 - 2: Initialize target networks θ'_1, θ'_2
 - 3: Initialize replay buffer $\mathcal{D}$
 - 4: **for** each iteration **do**
 - 5: **for** each environment step **do**
 - 6: Sample $a_t \sim \pi_\phi(\cdot|s_t)$
 - 7: Observe s_{t+1}, r_t
 - 8: Store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
 - 9: **end for**
 - 10: **for** each gradient step **do**
 - 11: Sample batch from $\mathcal{D}$
 - 12: Update Q-functions:
 $\theta_i \leftarrow \theta_i - \lambda_Q \nabla_{\theta_i} J_Q(\theta_i), \quad i \in \{1, 2\}$
 - 13: Update policy:
 $\phi \leftarrow \phi - \lambda_\pi \nabla_\phi J_\pi(\phi)$
 - 14: Update temperature:
 $\alpha \leftarrow \alpha - \lambda_\alpha \nabla_\alpha J(\alpha)$
 - 15: Update target networks:
 $\theta'_i \leftarrow \tau\theta_i + (1 - \tau)\theta'_i, \quad i \in \{1, 2\}$
 - 16: **end for**
 - 17: **end for**
-

IV. EXPERIMENTAL SETUP

A. Base Attitude Regulation Environment

The spacecraft is modeled as a rigid body with diagonal inertia matrix:

$$I = \text{diag}(100, 75, 80) \quad (17)$$

The environment is implemented using the Gymnasium framework, providing a standard interface for state, action, and reward interactions. The simulation uses a fixed timestep of $dt = 0.1$ s and a maximum episode duration of 120 s. Initial conditions for attitude and angular velocity are sampled uniformly within bounded regions.

The reward function penalizes attitude error, angular velocity, and control effort, with additional shaping terms and terminal bonuses to encourage convergence. Success is defined based on threshold values for $\|\sigma\|$ and $\|\omega\|$.

B. SAC Training Setup

The Soft Actor-Critic (SAC) controller is trained using the Stable-Baselines3 implementation over a total of 150,000 timesteps. The agent uses an `MlpPolicy` with fully connected actor and critic networks, each using two hidden layers with 64 units per layer. During training, SAC samples actions from its stochastic policy for exploration, while deterministic actions are used during evaluation for consistent performance comparisons.

To improve training stability, a modified training environment is used with adjusted timestep, episode duration, and reward weighting. These modifications reduce problem difficulty and allow the policy to learn effective control behavior more efficiently. In addition, episodes are terminated upon satisfying the success condition, defined by thresholds on attitude and angular velocity error. This early termination enables the agent to receive terminal rewards more frequently and reduces the length of solved trajectories, improving training efficiency.

Hyperparameter tuning for SAC is performed iteratively, beginning with standard baseline values and adjusting parameters such as reward weights, entropy coefficient behavior, and action scaling to improve convergence stability and final performance. Particular attention is given to reward shaping and entropy regularization to balance exploration and control accuracy.

C. LQR Evaluation Setup

The LQR controller is constructed using a linearized model of the spacecraft dynamics about the equilibrium point. The weighting matrices Q and R are initially selected using Bryson's rule [2], which normalizes each state and control input based on expected maximum values.

Following this initial selection, the weighting matrices are further tuned empirically to improve performance on the nonlinear system, particularly for larger initial condition deviations. The resulting controller is evaluated over 50 independent episodes using the nonlinear simulation environment.

No early termination is applied during evaluation, and performance metrics are computed over the full episode duration to assess convergence behavior and robustness.

D. Evaluation Metrics

Controller performance is evaluated using the following metrics:

- Mean episode return
- Standard deviation of return
- Final attitude error $\|\sigma\|$
- Final angular velocity $\|\omega\|$
- Success rate based on convergence thresholds

Both controllers are evaluated using identical dynamics, initial condition distributions, action limits, and success criteria to ensure a fair comparison.

V. RESULTS

A. LQR Performance

The performance of the LQR controller was evaluated over 50 independent episodes using the nonlinear spacecraft dynamics. Table I summarizes the aggregate performance metrics.

TABLE I
LQR EVALUATION RESULTS

Metric	Value
Mean Return	132438.49
Std Return	39723.64
Mean Final $\ \sigma\ $	8.58×10^{-5}
Mean Final $\ \omega\ $	4.31×10^{-5}
Success Rate	1.00

The LQR controller achieves a success rate of 100% across all evaluation episodes, with final attitude and angular velocity errors several orders of magnitude below the specified tolerances. This indicates consistent convergence to the desired equilibrium.

To illustrate the transient response, Figs. 1 and 2 show the evolution of the attitude error $\|\sigma\|$ and angular velocity $\|\omega\|$ for a representative episode.

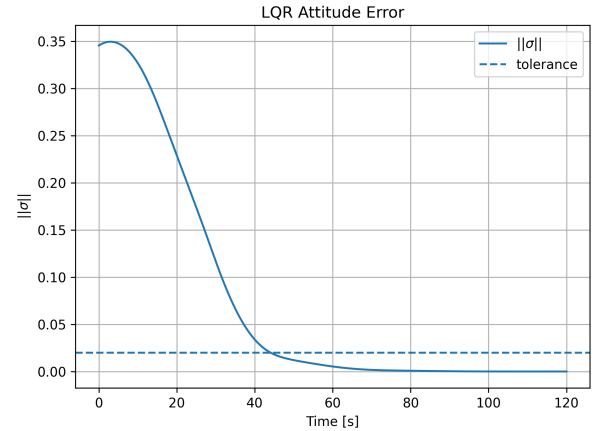


Fig. 1. Evolution of attitude error $\|\sigma\|$ under LQR control. The dashed line indicates the success tolerance.

The attitude error decreases monotonically after an initial transient phase and crosses the specified tolerance at approximately 40–50 seconds. Similarly, the angular velocity exhibits a brief transient increase before converging smoothly to zero. Both states remain within the tolerance bounds for the remainder of the simulation, indicating stable closed-loop behavior.

Overall, the LQR controller demonstrates reliable and repeatable convergence across all tested initial conditions, with low steady-state error and no observed instability.

B. SAC Performance

The Soft Actor-Critic (SAC) controller was trained for a total of 150,000 timesteps, corresponding to 1251 episodes. Table II summarizes the overall training performance.

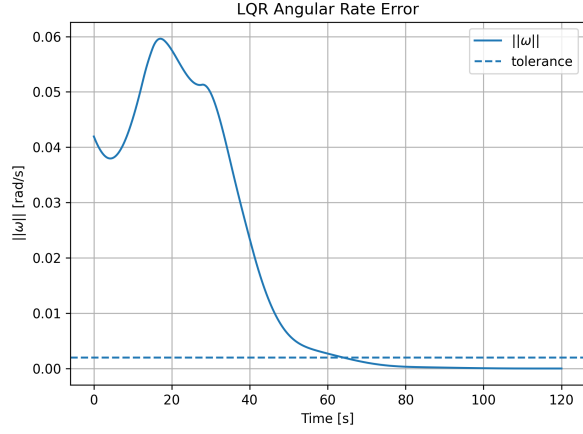


Fig. 2. Evolution of angular velocity $\|\omega\|$ under LQR control. The dashed line indicates the success tolerance.

TABLE II
SAC TRAINING RESULTS

Metric	Value
Total Episodes	1251
Mean Return	-675.46
Last 10 Episode Mean	-371.04

Fig. 3 shows the evolution of the episode return during training. The return improves steadily from approximately -2000 at early timesteps to values near -400 in later stages of training, indicating that the policy is learning to reduce attitude and angular velocity errors. However, significant variability is observed throughout training, particularly in the early and intermediate stages.

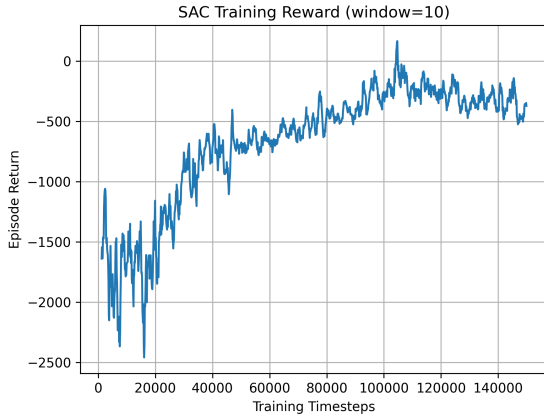


Fig. 3. SAC training reward over timesteps (moving average window of 10 episodes).

To further characterize learning behavior, Figs. 4 and 5 show the evolution of the final attitude and angular velocity errors over training. Both quantities decrease over time, with the attitude error reducing from values above 0.6 to approximately 0.03–0.05, and the angular velocity error decreasing to below

0.01 in later training stages.

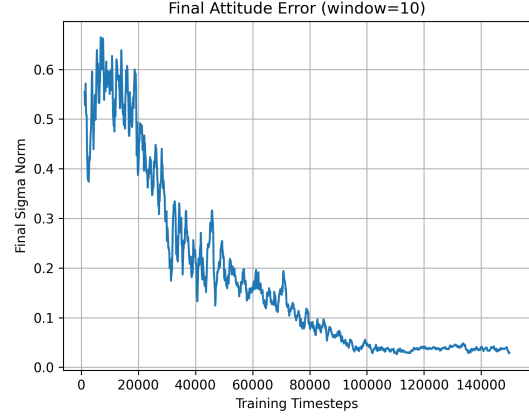


Fig. 4. Final attitude error $\|\sigma\|$ over training (moving average window of 10 episodes).

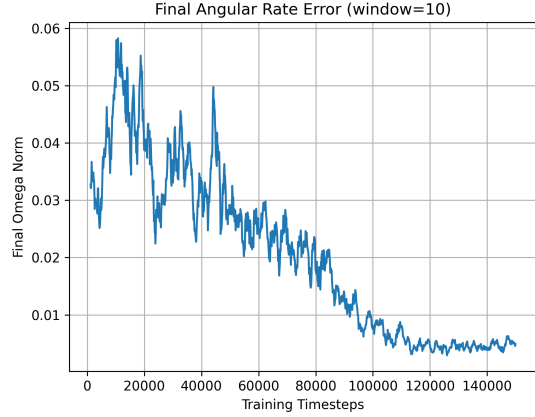


Fig. 5. Final angular velocity $\|\omega\|$ over training (moving average window of 10 episodes).

The success rate over training is shown in Fig. 6. Successful episodes begin to appear after approximately 90,000 timesteps, although the success rate remains low and intermittent throughout the remainder of training. This training success rate reflects the stochastic training policy and should be distinguished from deterministic evaluation performance.

Overall, the SAC controller demonstrates clear improvement in performance over training, as evidenced by increasing reward and decreasing state errors. However, convergence to consistently successful behavior is not achieved within the training horizon, and performance remains variable across episodes.

C. Controller Comparison

To directly compare controller performance, both LQR and SAC policies were evaluated on a common environment with identical dynamics, action limits, and termination conditions. Table III summarizes the aggregate performance metrics.

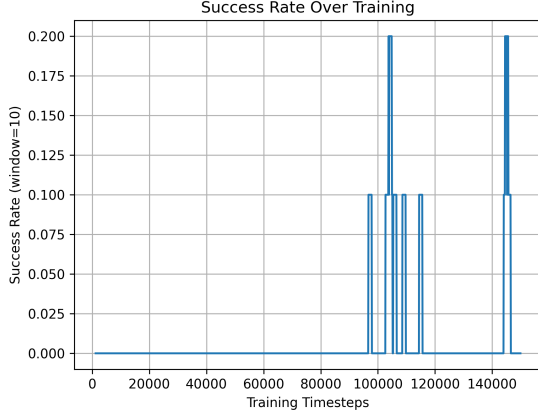


Fig. 6. Success rate over training (moving average window of 10 episodes).

TABLE III
CONTROLLER COMPARISON RESULTS

Metric	LQR	SAC
Mean Return	138305.01	85741.49
Std Return	35484.50	36428.41
Mean Final $\ \sigma\ $	4.7×10^{-5}	1.57×10^{-2}
Mean Final $\ \omega\ $	2.8×10^{-5}	3.34×10^{-4}
Success Rate	1.00	1.00
Mean Action Norm	0.568	0.521
Max Action Norm	1.692	1.494

Figs. 7, 8, and 9 show the distributions of final attitude error, angular velocity error, and return across evaluation episodes.

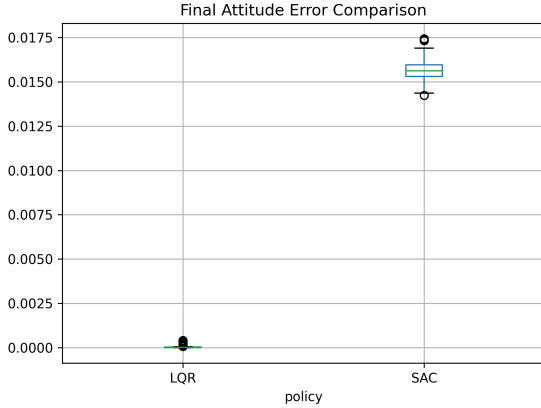


Fig. 7. Distribution of final attitude error $\|\sigma\|$ for LQR and SAC policies.

Both controllers achieve a success rate of 100% under the evaluation conditions, indicating that each is capable of satisfying the convergence criteria. However, the distributions of final state errors differ significantly. The LQR controller produces consistently low attitude and angular velocity errors, with tight clustering near zero. In contrast, the SAC policy exhibits higher variance and larger steady-state attitude errors, although angular velocity errors remain relatively small.

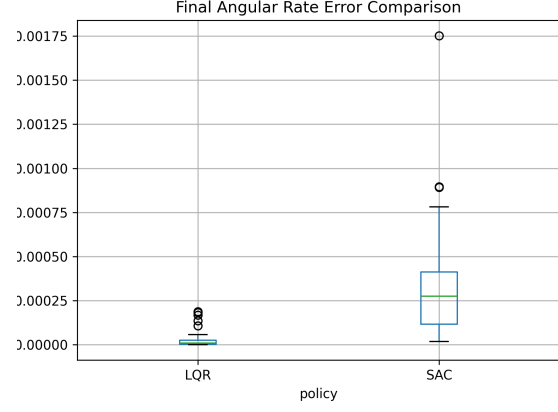


Fig. 8. Distribution of final angular velocity $\|\omega\|$ for LQR and SAC policies.

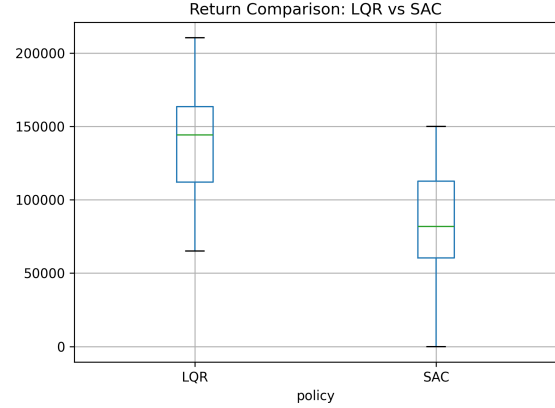


Fig. 9. Distribution of episode returns for LQR and SAC policies.

The return distributions reflect these differences, with LQR achieving higher mean return values and reduced variability compared to SAC. The action norms indicate that both controllers operate within similar control effort ranges, with LQR exhibiting slightly higher peak control magnitudes.

VI. DISCUSSION

The results demonstrate a clear distinction between the performance characteristics of the model-based LQR controller and the model-free SAC policy. While both controllers achieve a 100% success rate under the evaluation conditions, their transient behavior, steady-state accuracy, and consistency differ significantly.

The LQR controller exhibits highly consistent performance, with final attitude and angular velocity errors several orders of magnitude below the specified tolerances. This behavior is expected, as the controller is derived from a linearized model about the equilibrium point and directly minimizes a quadratic cost on the system state. As a result, LQR produces smooth and predictable convergence with minimal steady-

state error when operating within the region where the linear approximation is valid.

In contrast, the SAC policy demonstrates improved performance over training but does not achieve the same level of precision as LQR. Although the SAC controller consistently satisfies the success condition, the final attitude error remains significantly larger and exhibits greater variability across episodes. This indicates that while the learned policy captures the general control strategy required for stabilization, it does not fully refine the control action to minimize residual error.

Several factors contribute to this behavior. First, the reward function emphasizes convergence to within tolerance rather than strict minimization of the state error. Once the policy consistently reaches the success region, there is limited incentive to further reduce the error magnitude. Second, early termination of episodes upon success during training reinforces this behavior, as trajectories are truncated before the policy experiences the benefit of further error reduction. Third, the stochastic nature of SAC introduces variability in the learned policy, which can lead to small steady-state deviations even after convergence.

The training dynamics of SAC further highlight the challenges associated with reinforcement learning in this setting. While the reward improves steadily and state errors decrease over time, the success rate remains intermittent and does not fully stabilize within the training horizon. This suggests that the policy has not fully converged to a stable optimum, and may require additional training, improved reward shaping, or modified hyperparameters to achieve more consistent performance.

An additional distinction arises between SAC training and evaluation behavior. During training, the policy is stochastic, with actions sampled from a probability distribution to encourage exploration. This results in variability in both the observed reward and success rate, even as the underlying policy improves. In contrast, evaluation is performed using a deterministic policy, typically by selecting the mean action. This reduces variability and leads to more consistent convergence behavior. As a result, the evaluation performance of SAC appears more stable than the training metrics would suggest. Additionally, entropy regularization further amplifies this effect during training by promoting exploration, which can temporarily degrade performance metrics despite improving the overall policy.

In addition to state convergence, the control effort required by each controller provides further insight into their behavior. The LQR controller explicitly penalizes control input through the quadratic cost term $u^T R u$, enforcing a fixed tradeoff between state error and control effort. In contrast, the SAC policy incorporates control cost through reward shaping, where the penalty on control input competes with other reward components and the entropy regularization objective. This can lead to slightly reduced average control effort, particularly near convergence, although both controllers operate within similar overall control ranges. The observed differences suggest that

SAC implicitly learns a balance between effort and convergence, while LQR enforces this tradeoff directly through its cost formulation.

Despite these limitations, SAC demonstrates the ability to learn a stabilizing control policy directly from interaction with the nonlinear system, without requiring an explicit model or linearization. This highlights the flexibility of reinforcement learning methods, particularly in scenarios where system dynamics are complex or partially unknown. However, in problems where accurate models are available and the operating regime is well-defined, model-based controllers such as LQR remain highly effective and offer superior precision and reliability.

Overall, the comparison illustrates a fundamental tradeoff between model-based and learning-based control approaches. LQR provides strong performance with minimal tuning when a suitable model is available, while SAC offers adaptability and model-free learning at the cost of increased variability and reduced steady-state accuracy.

VII. CONCLUSION

This work compared a model-based Linear Quadratic Regulator (LQR) and a model-free Soft Actor-Critic (SAC) controller for spacecraft attitude stabilization. Both approaches successfully achieved the control objective, demonstrating the ability to drive attitude and angular velocity errors below specified tolerances.

The LQR controller exhibited highly consistent performance, achieving near-zero steady-state error with minimal variability across episodes. This reflects the effectiveness of model-based optimal control when an accurate system model and operating point are available. In contrast, the SAC controller demonstrated the ability to learn a stabilizing policy directly from interaction with the nonlinear environment, without requiring explicit knowledge of the system dynamics. However, the learned policy exhibited larger steady-state errors and greater variability, indicating incomplete convergence within the training horizon.

The results highlight a key tradeoff between the two approaches. LQR provides precise and reliable control with minimal tuning when a suitable model is available, while SAC offers flexibility and model-free learning at the cost of reduced precision and increased training complexity. Differences in performance were influenced by reward design, training dynamics, and the stochastic nature of reinforcement learning.

Future work may explore improved reward formulations, longer training horizons, or hybrid approaches that combine model-based structure with learning-based adaptation. Such methods could leverage the strengths of both approaches to achieve robust and high-performance control across a wider range of operating conditions.

CONTRIBUTION STATEMENT

This project was completed individually by Kirin Kawamoto, who was responsible for all aspects of the work, including implementation, experimentation, analysis, and report preparation.

REPORT AVAILABILITY

The author grants permission for this report to be posted publicly.

IMPLEMENTATION DETAILS

The following components were implemented or used off-the-shelf:

- **LQR Controller (Implemented):** The LQR controller was implemented from first principles, including linearization of the spacecraft attitude dynamics and solution of the continuous-time algebraic Riccati equation to compute the feedback gain matrix.
- **Environment (Implemented):** A custom Gymnasium environment was developed to simulate nonlinear rigid-body attitude dynamics using Modified Rodrigues Parameters (MRPs) and angular velocity states. This includes the system dynamics, reward function, action scaling, and termination conditions.
- **SAC Controller (Off-the-Shelf):** The Soft Actor-Critic algorithm was used via the Stable-Baselines3 library. The core SAC algorithm, including policy and value function updates, was not implemented from scratch.
- **Training and Evaluation (Implemented):** The training pipeline, environment configuration, model selection, and evaluation procedures were implemented for this project, including custom training settings and performance analysis.

REFERENCES

- [1] R. E. Kalman, "Contributions to the theory of optimal control," *Boletín de la Sociedad Matemática Mexicana*, vol. 5, pp. 102–119, 1960.
- [2] A. E. B. Jr. and Y.-C. Ho, *Applied Optimal Control: Optimization, Estimation, and Control*. Hemisphere Publishing, 1975.
- [3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 1861–1870. [Online]. Available: <https://proceedings.mlr.press/v80/haarnoja18b.html>
- [4] B. Gaudet and R. Furfaro, "Deep reinforcement learning for spacecraft attitude control," *Acta Astronautica*, 2020.
- [5] J. Buchli *et al.*, "Learning control policies for robotics using reinforcement learning," *IJRR*, 2009.
- [6] M. Stephenson and H. Schaub, "Bsk-rl: Modular, high-fidelity reinforcement learning environments for spacecraft tasking," in *International Astronautical Congress*, Milano, Italy, Oct. 14–18 2024.
- [7] H. Schaub and J. L. Junkins, *Analytical Mechanics of Space Systems*, 4th ed. Reston, VA: AIAA Education Series, 2018.
- [8] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>